Custom UDP Server vs. Godot ENet: A Cloud Benchmark of Multiplayer Server Capacity

Carlos Nunez
Department of Computer Science and Engineering, Shippensburg University
Mentor: Dr. Girard
CSC 499 – Senior Research Methods

Fall 2026 (draft)

Abstract

Developers who build multiplayer games in Godot must choose between the engine's built-in ENet dedicated server and a custom server written outside the engine. We compare a custom C++ UDP server with Godot's ENet headless server under the same workload, NodeStrike, a 20×20 grid game in which every client is a bot that moves one tile per 20 Hz server tick. Both servers run on Google Cloud with one server VM and three client VMs. Two tests are used: a progressive ladder that adds clients from 1 to 300, run eight times per backend, and a 20-minute hold at fixed loads of 180 to 300 clients. We measure latency, packet handling (snapshots received per client per second), client limit, and server tick rate. TODO: write the results sentences from the granular ladder runs (the hold tests are still to be run).

Keywords: game networking, UDP, ENet, Godot, tick rate, server capacity, benchmarking

Draft note. The ladder results in Section 8.1 are the eight runs of the new granular batch (both patches applied, larger client VMs). The abstract and the hold results are still from the first round of runs (**superseded**); the hold tests have not been rerun yet. See Section 7 for the two comparability problems that were fixed.

1 Introduction

TODO: write the introduction (problem, what this paper does, the questions).

2 Background and Related Work

TODO: write the background (servers and transport, related work).

2.1 Ticks and snapshots

TODO: write what a tick and a snapshot are.

Term	Meaning in NodeStrike
Hz (hertz)	Times per second. NodeStrike runs at 20 Hz, i.e. 20 ticks per second.
Tick	One server cycle: read incoming client updates, update the world, send a snapshot. Fixed at 50 ms (1000/20).
Snapshot	A capture of the game state at one tick: every player's position at that instant. The server sends it to each client ($16 + 12N$ bytes for N players, split into datagrams of at most 1,200 bytes). Each arrival updates the client's current ping reading and gap count.
Latency / ping	How long data takes to travel. Round-trip time (RTT) is there and back. In the game client, ping is the time from sending its last position update until the next snapshot arrives, so it includes waiting for the server's next 50 ms tick.
Gap / loss	Each snapshot has a sequence number; a missing number means a snapshot was lost or dropped.
Effective tick rate	Ticks the server actually completes per second. Healthy $\approx$ 20 Hz; below 18 Hz is treated as overloaded.

Table 1: Terms used in this paper.

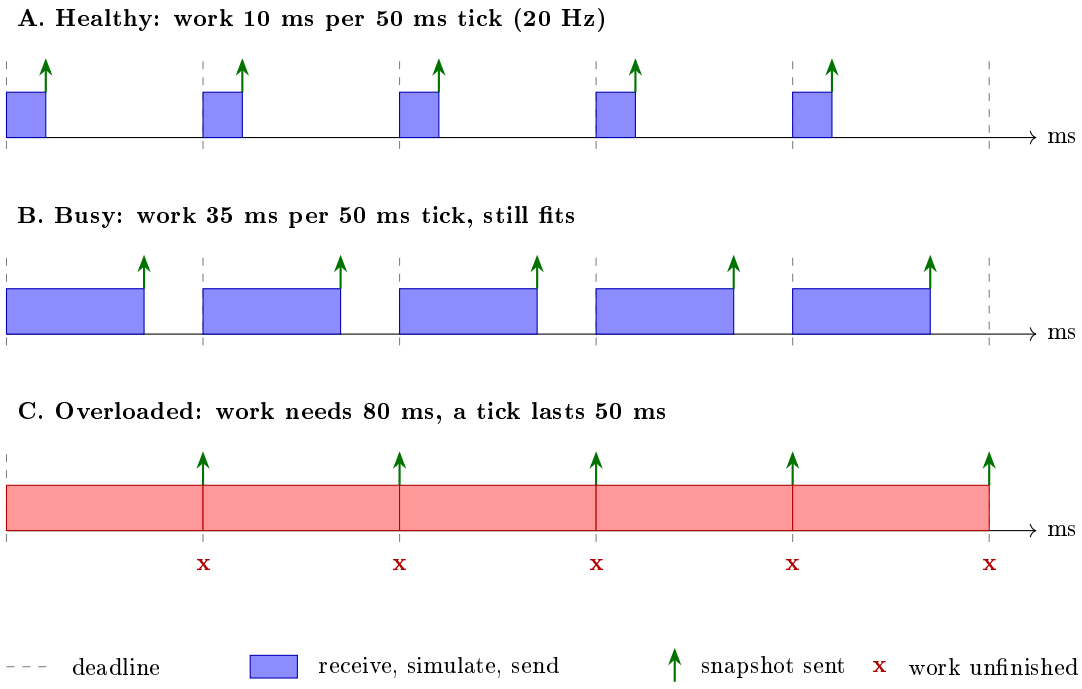


Figure 1: Schematic of a server tick (illustrative timings, not measured data). Each tick the server receives client input, updates the world, and sends a snapshot; at 20 Hz it has 50 ms for all of it. If the work per tick grows with the number of clients and passes 50 ms (panel C), the snapshot still leaves at the 50 ms deadline, because the next tick's work has to start. The tick rate therefore stays near 20 Hz, but the work is unfinished, so the snapshots carry less up-to-date information and clients see worse ping and more gaps.

3 Making of the Godot Client and ENet Server

TODO: write how the Godot client and the Godot ENet server were built (goals, design decisions, problems you hit).

Notes from the code (delete when written).

- One Godot 4.7 project, NodeStrike (20×20 grid). One script, `main.gd` (about 860 lines), runs every role.
- The role is chosen with command-line flags: `-transport=udp|enet`, `-role=client|server`, `-address`, `-port`, `-udp-only`, `-enet-snapshot=binary|variant`.
- Client: spawns a bot, picks a goal tile, moves one tile per tick from a 20 Hz timer, and sends its position every tick (an 18-byte message over UDP, or a remote procedure call over ENet).
- Client default is the C++ UDP server; a windowed client falls back to ENet after 8 s with no reply, unless `-udp-only` is set.
- ENet server mode: creates an ENet multiplayer peer (up to 2048 clients), runs a 20 Hz timer, drops clients silent for more than 5 s, and each tick sends one snapshot to every peer by remote procedure call (compact binary, or a list of dictionaries).
- Headless mode (no window) is used for the benchmark and prints ping, packets sent and received, and gaps in snapshot numbers.
- Two small extra scripts, `enet_server.gd` and `enet_client.gd`, are a simpler ENet pair that the cloud benchmark does not use.

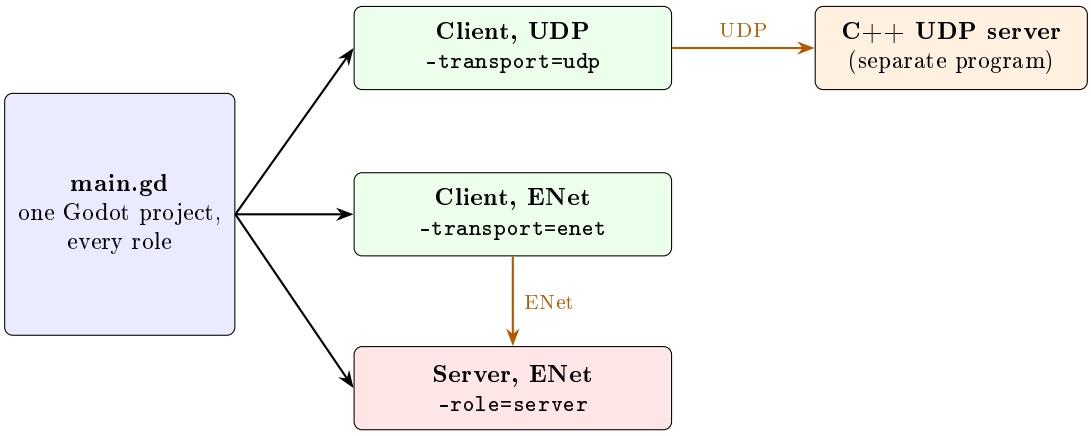


Figure 2: The Godot project. One script is started in different modes. A UDP client talks to the separate C++ server; an ENet client talks to the same project started as an ENet server.

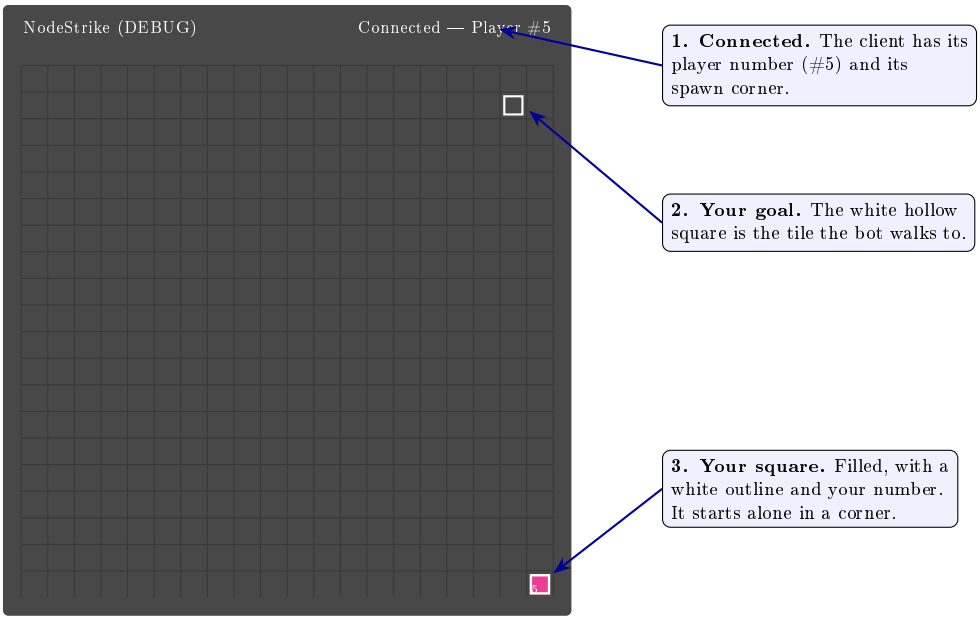


Figure 3: The Godot client the moment it joins (a redrawing of the NodeStrike window). The board is the 20×20 grid; the only player in view is the client's own.

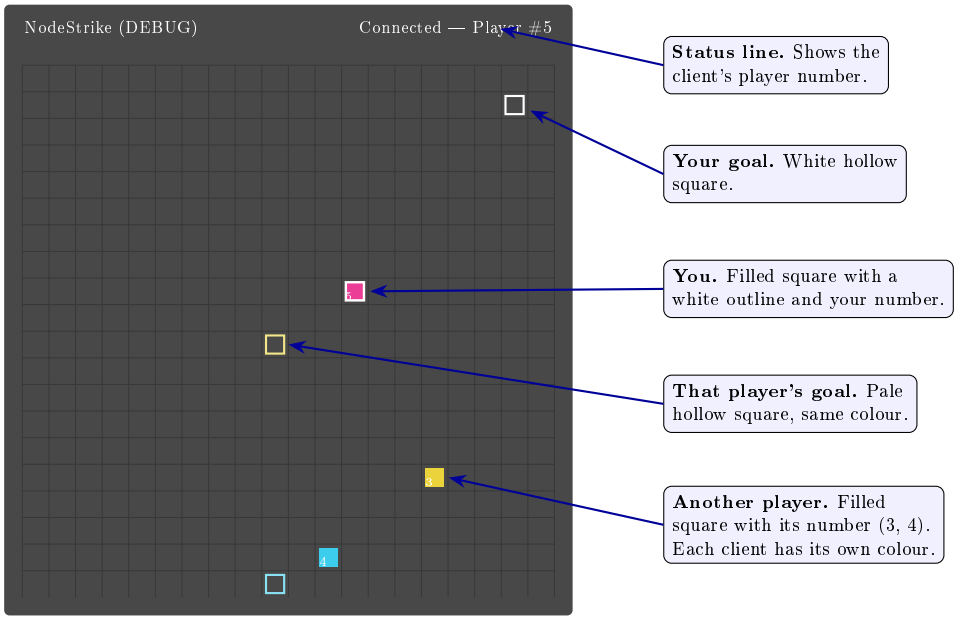


Figure 4: The running NodeStrike client (redrawn from a screenshot of a client started in a window; headless benchmark clients draw nothing). Each tick every bot moves one tile toward its goal. The server's snapshot tells each client where every other player is; the client only draws it. Each player's pale hollow square is that player's goal.

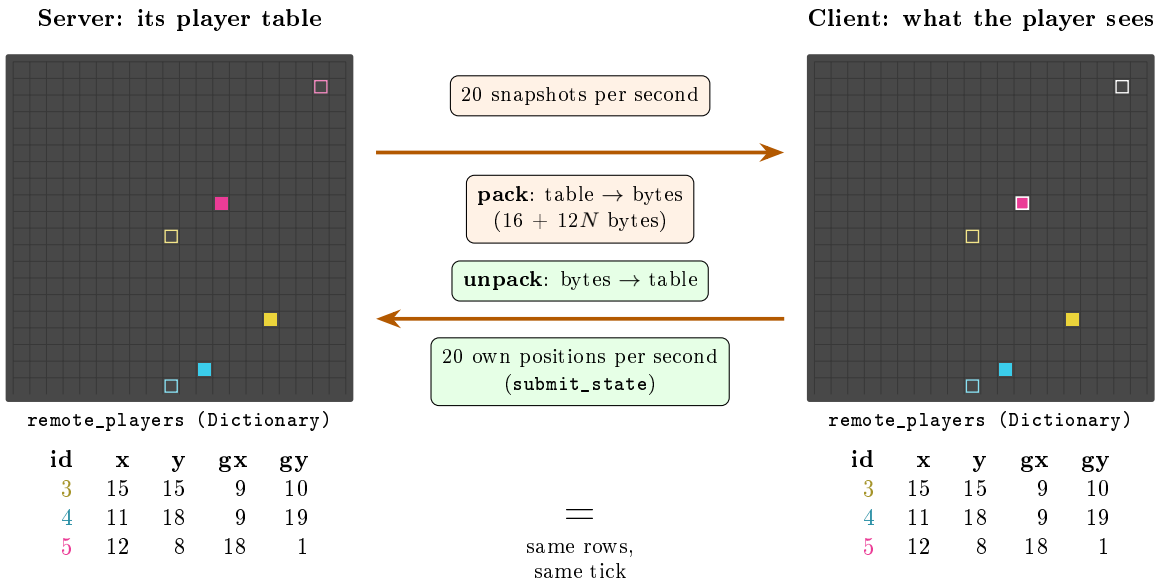


Figure 5: Server and client side by side (redrawn). The server keeps every player in a table (a Godot Dictionary, id → position and goal). Every tick it packs the table into bytes and sends it. Each client unpacks the bytes into its own table of the same name and draws it, so the two screens show the same players and follow each other, about one network trip apart. The client's only extra is the white outline on its own square. Going the other way, each client sends only its own row.

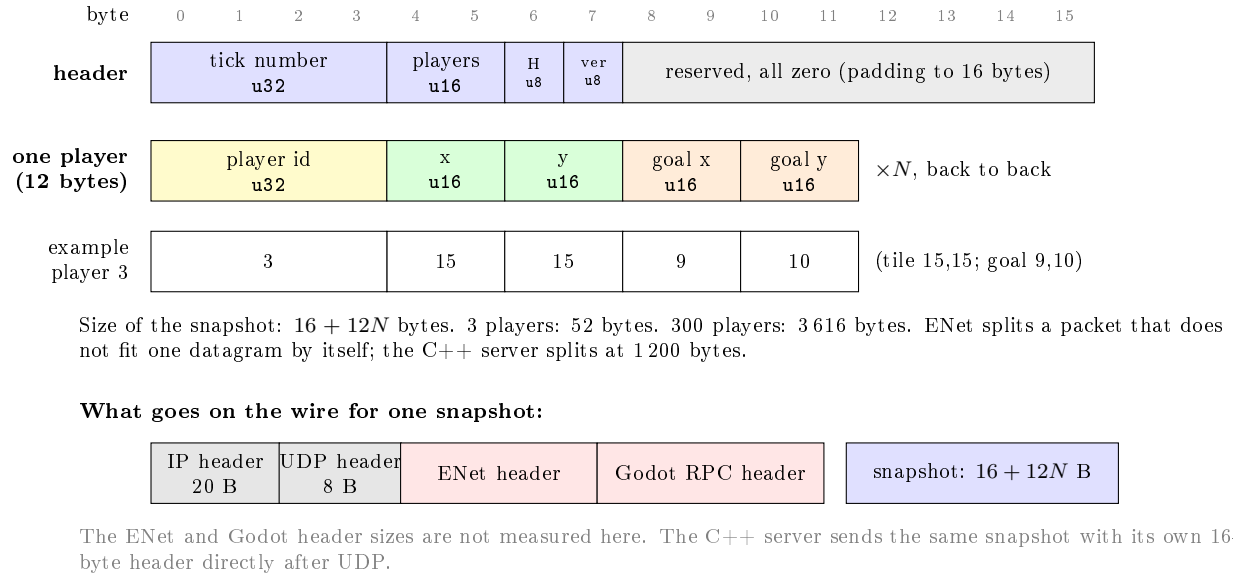


Figure 6: How one snapshot is packed (ENet backend, default binary format). The server writes the header and then one 12-byte block per player into one byte array with `encode_u32`/`encode_u16` and sends it as a remote procedure call. The client reads the same positions back with `decode_u32`/`decode_u16`. The alternative `-enet-snapshot=variant` sends the same rows as a list of dictionaries, with the keys `id, x, y, gx, gy` sent as text each time.

[DRAFT] The snapshot code.

```
func _broadcast_binary_enet_tick() -> void:
    var player_ids := remote_players.keys()
    var player_count := mini(player_ids.size(), 0xffff)
    var packet := PackedByteArray()
    packet.resize(16 + player_count * 12)
    packet.encode_u32(0, tick_count)        # bytes 0-3: tick number
    packet.encode_u16(4, player_count)      # bytes 4-5: players
    packet[6] = 1 if h_overlay_active else 0
    packet[7] = ENET_SNAPSHOT_VERSION       # byte 7: version
    for i in range(player_count):           # bytes 8-15 stay 0
        var pid := int(player_ids[i])
        var player: Dictionary = remote_players[pid]
        var offset := 16 + i * 12
        packet.encode_u32(offset, pid)
        packet.encode_u16(offset + 4, int(player.get("x", 0)))
        packet.encode_u16(offset + 6, int(player.get("y", 0)))
        packet.encode_u16(offset + 8, int(player.get("gx", 0)))
        packet.encode_u16(offset + 10, int(player.get("gy", 0)))
    rpc("receive_binary_snapshot", packet)
```

Figure 7: **[DRAFT]** The ENet server packs the player table into one byte array and sends it to every client (excerpt of `main.gd`, constants written out as numbers and the statistics counter left out).

4 Making of the C++ UDP Server

TODO: write how the C++ server was built (goals, design decisions, problems you hit).

Notes from the code (delete when written).

- One file, `server/main.cpp` (about 590 lines), C++17, plain POSIX sockets, one thread.
- Binary protocol, version 2 (Table 2). A player is identified by its network address, not by anything the client sends.
- Main loop (Figure 8): wait for a packet (until the next tick is due, at most 5 ms), read everything waiting, and when the next tick is due, run it.
- A tick drops clients silent for more than 5 s and sends a snapshot to every client, split into datagrams of at most 1,200 bytes.
- Once a second it prints one telemetry line (tick number, players, packets and bytes in and out); the benchmark records these lines.
- A separate terminal display, `tui/tui.cpp`, exists for watching a server.

Message	Direction	Size	Contents
JOIN	client to server	2 B	type, version
ACK	server to client	6 B	type, version, player id
STATE	client to server	18 B	position, timestamp, number, goal
TICK (snapshot)	server to client	16 + 12N B	header, then id, position and goal for each of N players

Table 2: Messages of the C++ server protocol (version 2). Snapshots larger than 1,200 bytes are split into several datagrams.

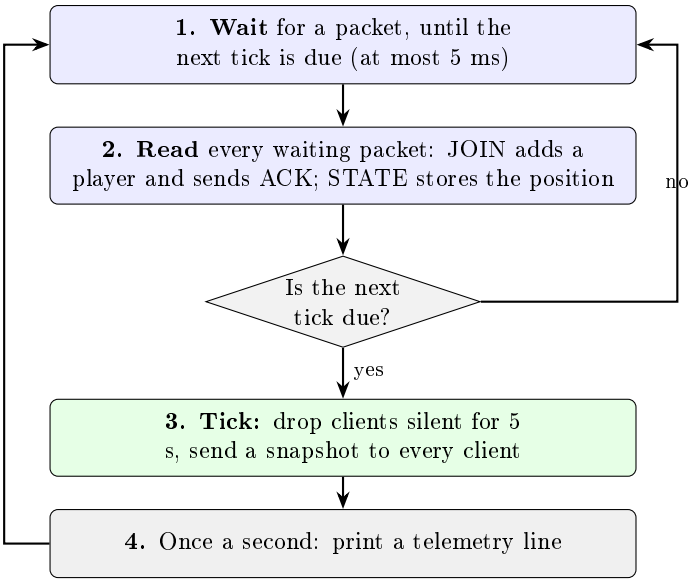


Figure 8: Main loop of the C++ server. It repeats forever on one thread.

5 How Both Servers Work: The Ins and Outs

TODO: write how the two servers work and how they differ.

	C++ UDP server	Godot ENet server
Networking	raw UDP datagrams	ENet through Godot's multiplayer peer
Default port	7777	7778
Tick timing	fixed-timestep schedule: each tick is due at an exact multiple of 50 ms, so it runs at 20 ticks per second (the first version restarted its timer at each tick and drifted to about 19, see Section 7)	Godot timer that fires every 50 ms and keeps its timetable (20 ticks per second)
Join	JOIN message, ACK with player id	connection handled by ENet; the peer id is the player id
Position update	18-byte STATE datagram (position, timestamp, number, goal), sent every tick	`submit_state` remote procedure call (unreliable) with the same fields, sent every tick
Snapshot	16 + 12N bytes, split into datagrams of at most 1,200 B	16 + 12N bytes (binary format, header padded from 8 to 16 bytes so both sizes match), one remote procedure call
Client eviction	silent for 5 s	silent for 5 s
Latency measure	game client's ping: last send until next snapshot	same Godot client script, same measure
Telemetry	one line per second	periodic server statistics

Table 3: The two backends side by side.

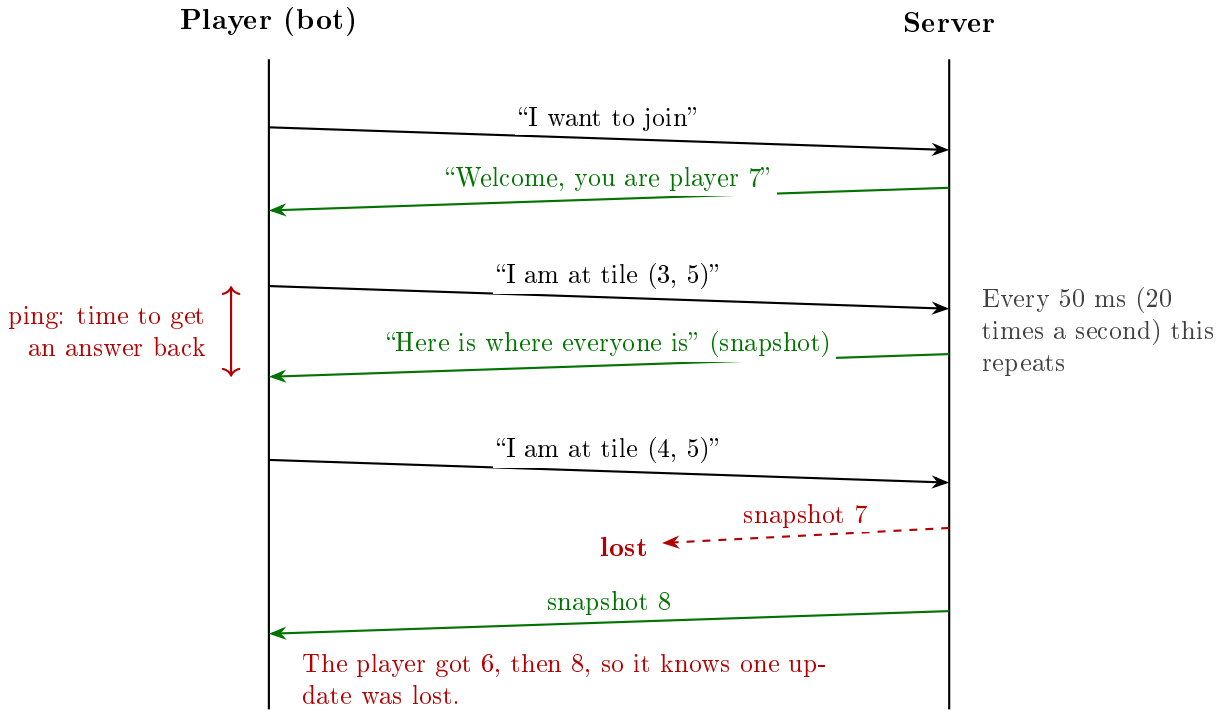


Figure 9: What the player and the server say to each other. The player says where it is, the server replies with where everyone is, and this repeats 20 times a second. Ping is how long it takes to hear back; a lost reply shows up as a missing number. The ENet backend does the same thing through Godot's own networking.

5.1 One tick, step by step

Every 50 ms the server does the same five things, in this order. If steps 1 to 4 take less than 50 ms, it waits for the next tick. If they take longer, the next tick starts late.

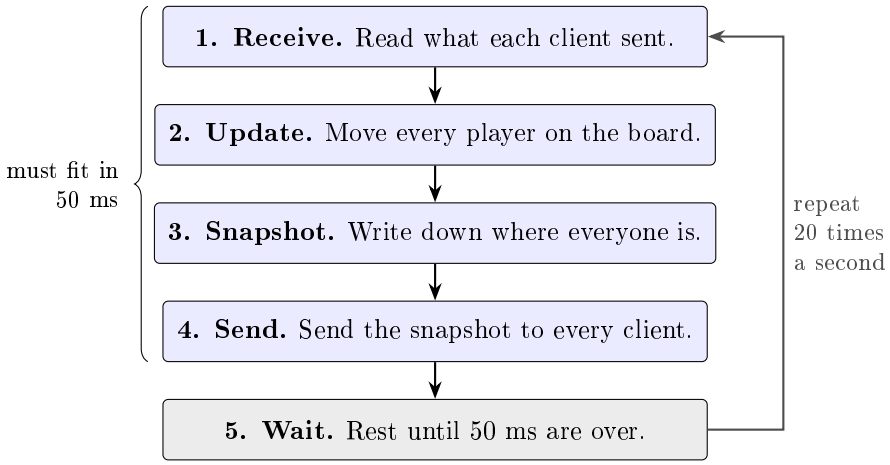


Figure 10: The server tick. Steps 1 to 4 are the work; step 5 is spare time. The more clients there are, the longer steps 3 and 4 take.

5.2 Why more clients means much more work

Each snapshot lists every player, and every client gets one. So when the number of clients N doubles, each snapshot gets about twice as big and there are twice as many to send: the work grows about four times. Figure 11 shows the amount of data this means for the C++ server (snapshot of $16 + 12N$ bytes sent to N clients, 20 times a second).

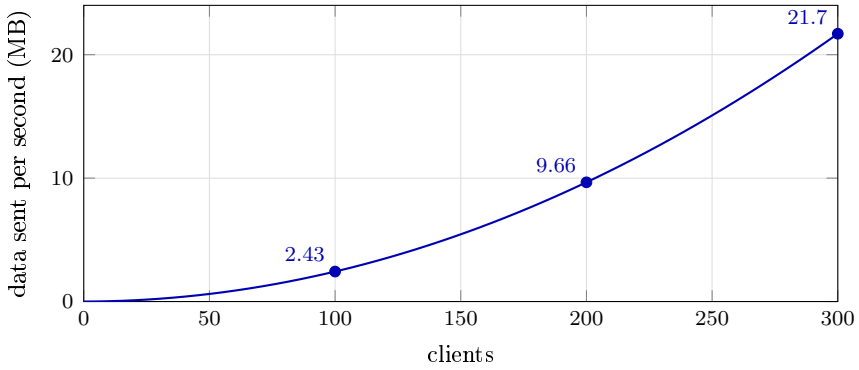


Figure 11: Data the server must send each second as clients are added (calculated from the packet format; ENet is similar). Tripling the clients from 100 to 300 multiplies the data about nine times.

5.3 How clients end up responding less (and recovering)

A client counts as *responding* if it received at least one new snapshot during the measurement period. While the server keeps up, every client gets a snapshot every tick. Once the work per tick

is too much, snapshots arrive late, get skipped, or pile up behind each other, and some clients stop receiving any. Figure 12 shows the chain of events, and Figure 13 shows what one client sees.

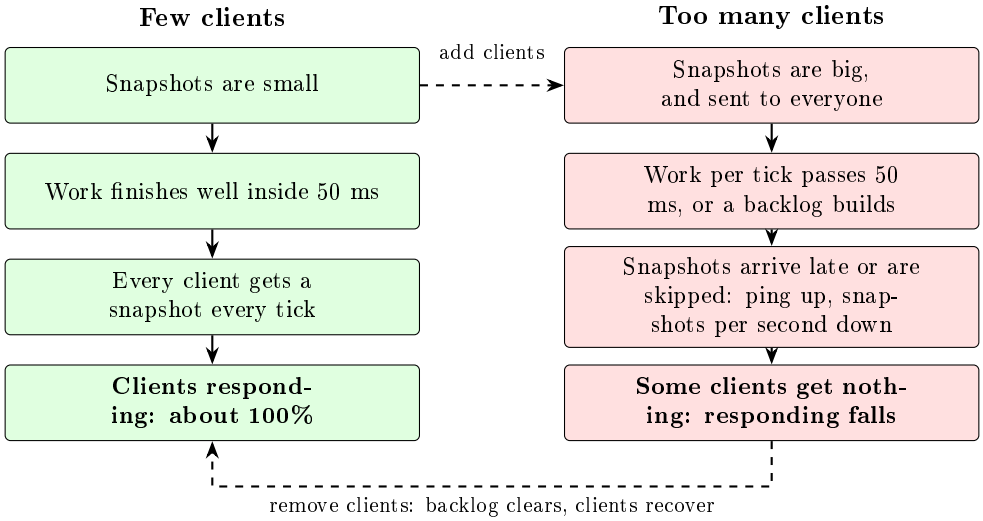


Figure 12: Left: a server that keeps up. Right: how adding clients leads to fewer clients responding. The dashed arrows show that removing load reverses it.

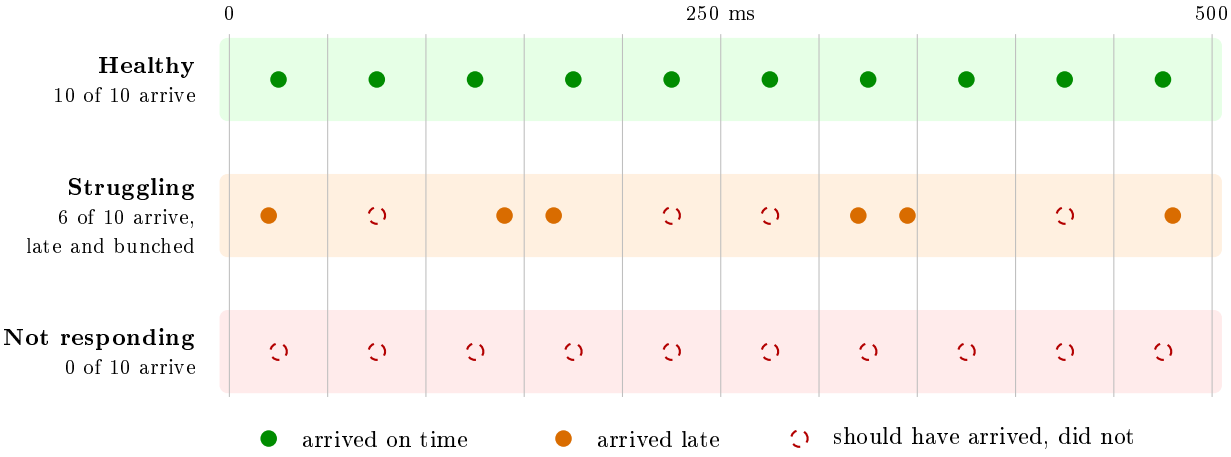


Figure 13: What one client receives over 500 ms (schematic). Each dot is a snapshot the server should send every 50 ms. Healthy: all arrive on time. Struggling: some come late and bunched together, others never arrive. Not responding: none arrive, so the client is counted as not responding.

6 Experimental Method

All runs use one server VM and three client VMs on Google Cloud (the server on an e2-standard-2 machine, the clients on n2-standard-4 machines). ENet snapshots use the binary format from 22 September onward.

6.1 Main measures

Three measures are the main focus of the study. The rest of the data (CPU, memory, network use, server counters and every other column collected) is kept and still matters: it is used to explain why the main measures change. A tick-rate drop is the warning sign that one of the three is failing.

Measure	Plain meaning	On the charts
Latency	How long data takes to go to the server and back, in milliseconds (ping). Lower is better.	Ping in milliseconds (log scale)
Packet handling	How many packets the server receives and sends each second without losing or delaying them. A healthy client gets one snapshot per tick, about 20 per second.	Snapshots received per client per second (20 is perfect)
Client limit	The largest number of clients the server supports before performance degrades.	Share of clients still responding; where the lines break
Tick rate (warning sign)	Ticks the server completes per second. 20 Hz is healthy; below 18 Hz means the server is falling behind.	Server ticks per second, with an 18 Hz line

Table 4: The three main measures, and the tick-rate warning sign. All other collected data supports them.

6.2 Two kinds of test

We run two kinds of test, because they answer different questions.

- **Progressive (granular) ladder.** Start with one client and add clients one step at a time, holding each step for a short time (10 s in the October runs). The state list is 1 client, then 10, 11, 12, up to 300: 292 states per backend. Question: at what client count does the server stop keeping up?
- **Hold.** Ramp up to a target client count (targets 180, 260, 270 and 300), stop adding clients, and keep them connected for 20 minutes, measured in 120 windows of 10 s. Question: does the server stay stable at a fixed load?

6.3 How the data is collected

Figure 15 shows where a test runs. A Python script automates every run:

1. It creates one server VM and three client VMs on Google Cloud.
2. It starts the server (the C++ UDP server, or the Godot ENet server in headless mode).
3. It gives each client VM a plan: a list of states, each with a total client target and a duration. The client VMs start bot clients so that the total matches the plan, and each bot plays the NodeStrike game. Every bot is the same Godot client (`main.gd`): in UDP mode it talks to the C++ server and in ENet mode to the Godot server, so both servers are driven by the same client code.

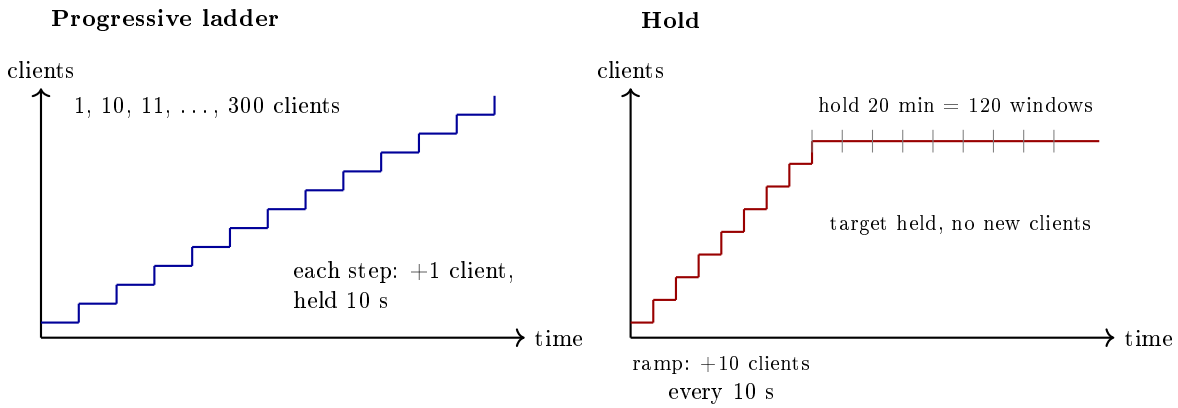


Figure 14: The two kinds of test (schematic). The ladder keeps adding clients and shows where performance breaks. The hold stops at a target and shows whether performance stays steady.

4. During every state, each client counts what it sent and received, checks the snapshot sequence numbers for gaps, and measures ping about once per second.
5. The server writes one telemetry row per second (tick count, players connected, packets and bytes in and out). Each VM also records CPU, memory and network use every second.
6. When the run ends, all files are downloaded to this computer, the VMs are deleted, and analysis scripts merge the repetitions and apply pass/fail thresholds.

Gaps and incomplete snapshots are indicators of protocol-level loss, not measured network packet-loss percentages.

6.4 What the charts show

Every chart plots the same four things for both backends:

- **Latency:** average ping.
- **Packet handling:** snapshots received per client per second.
- **Clients responding:** how many clients are still getting answers.
- **Tick rate:** server ticks per second.

On the ladder charts the horizontal axis is the number of clients, so each point is one client count. On the hold charts it is time, so each point is one 10 s window.

6.5 Health criteria and caveats

TODO: write the pass/fail limits and what the measurements can and cannot show.

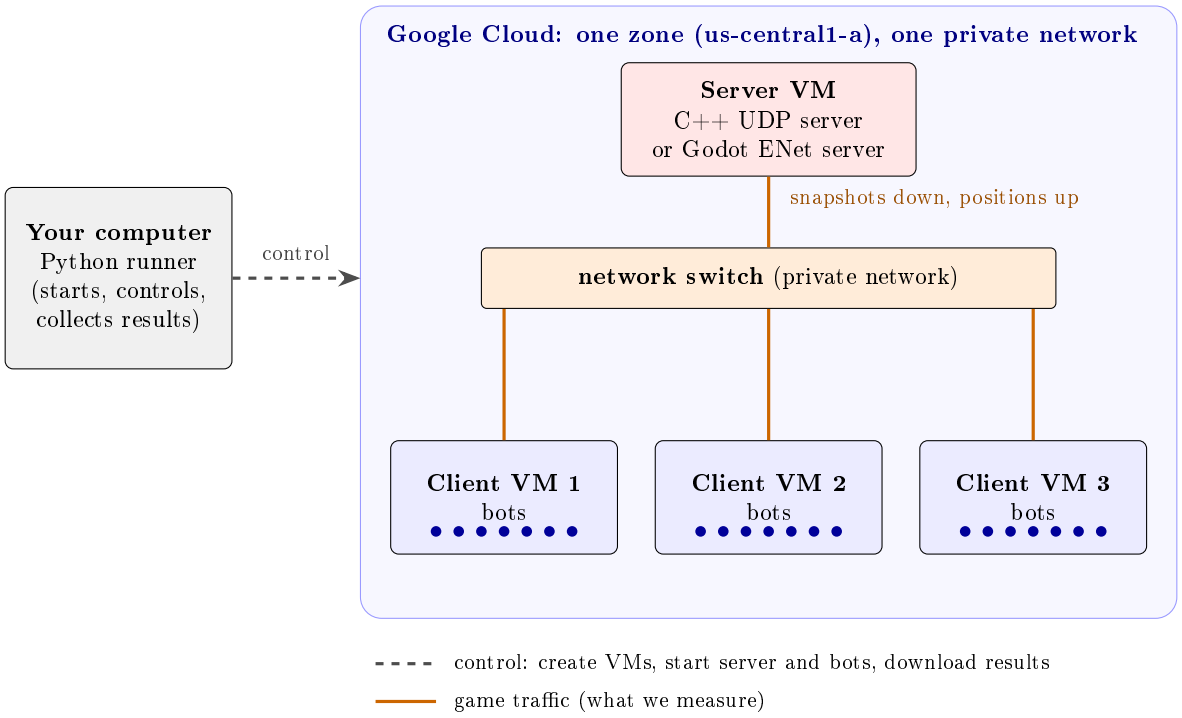


Figure 15: Where a test runs. The runner on your computer creates one server VM and three client VMs, starts the server, tells each client VM how many bots to run, and downloads the results afterwards. The bots and the server talk to each other over the cloud's private network (a switch). Each client VM runs about a third of the bots, so the server sees all of them as one crowd.

6.6 Runs

Granular ramps with $N_{\max} = 300$ give 292 states per backend (1 plus 10 through 300).

Date (2026)	Type	Backend	Counts	Time/state	Runs	Statu
Oct 23–24	Progressive ladder (granular)	C++ server, ENet server	1, 10–300	10 s	8 runs	ok
Not yet run	Hold	C++ server, ENet server	180, 260, 270, 300	20 min hold	—	TOD

Table 5: Runs used in the results. The progressive ladder is eight complete runs per backend, each run a separate pass from 1 to 300 clients (one repetition each). The C++ server is driven by Godot UDP clients and the ENet server by Godot ENet clients. The hold tests have not been run yet.

7 Problems Found in the First Runs, and How They Were Fixed

After the first complete set of runs we found two problems that made the comparison between the C++ and the ENet server unfair, plus some measurement quirks. This section explains, for each one, what it was, why it happened, how it was fixed, and what it means for the first-round data. The first-round results in Section 8 are kept for reference and are superseded: new data with the patches below applied will be made (8 progressive-ladder runs and 8 hold runs per backend) and will replace them.

7.1 Overview

Problem	Why it mattered	Fix	Status
C++ ticks drifted to about 19 Hz	C++ did about 5% fewer ticks per second than ENet	Fixed-timestep tick schedule	Fixed
Snapshot sizes differed (8 vs 16 header bytes)	ENet sent slightly smaller packets than C++	ENet header padded to 16 bytes	Fixed
Slow ENet states and a last-state artifact	ENet rates were measured over longer windows; the last ladder state was inflated	To be handled in the analysis	Noted

Table 6: The problems found in the first runs.

7.2 Problem 1: the C++ tick drifted below 20 Hz

7.2.1 What we saw

Both servers are meant to tick at 20 Hz. In the first round the ENet server ticked at about 20 Hz, but the C++ server ticked at about 19.4 Hz with few clients and fell to about 18.9 Hz at 300 clients. Over a second that is roughly 5% fewer ticks and snapshots, so C++ was doing slightly less work than ENet.

7.2.2 Why it happened: the stopwatch

The C++ loop starts a tick as soon as it notices that 50 ms have passed, and then restarts its timer from that moment. If it notices 2 ms late, the next 50 ms are counted from the late moment, so the delay is never made up and each tick is 52 ms after the last. In one second that is $1000/52 \approx 19.2$

ticks instead of 20. With more clients each pass through the loop does more work, so the check for the next tick comes later and the drift grows. In the measured data the tick falls from about 51.5 ms (about 1.5 ms late) with few clients to about 52.9 ms (about 2.9 ms late) at 300 clients. The ENet server uses a Godot timer, which keeps its timetable, so it does not drift.

7.2.3 The drift in one picture

Figure 16 counts the ticks of both servers side by side. Each dot is one tick and the number is the tick count.

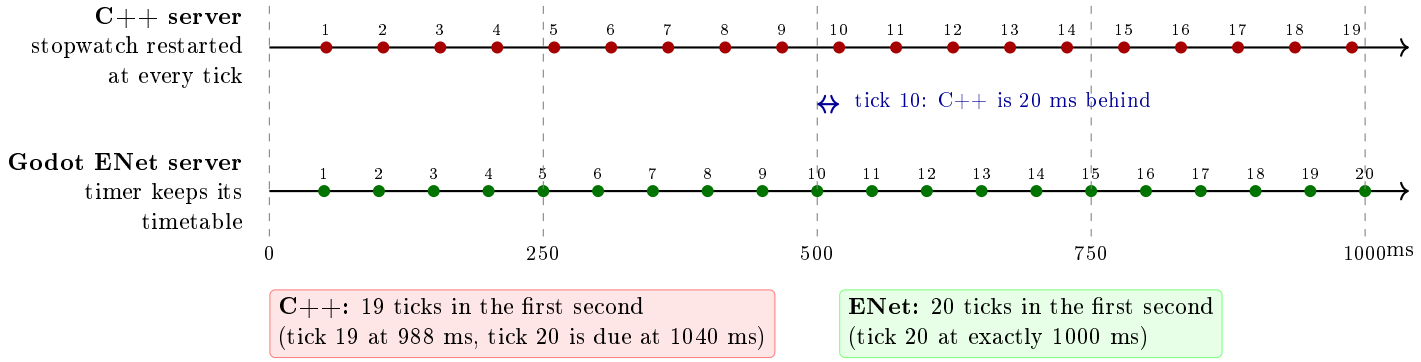


Figure 16: Counting ticks, C++ against ENet (schematic, each tick noticed 2 ms late). Every dot is one tick and the number is the tick count. The C++ loop restarts its stopwatch at each tick, so its ticks are 52 ms apart and it has counted only 19 ticks after one second. The Godot timer keeps its 50 ms timetable, so its tick 20 arrives at exactly 1000 ms. After 10 ticks the C++ server is already 20 ms behind, and the gap keeps growing.

7.2.4 A worked example

Say every tick is noticed 2 ms late.

- **Stopwatch (C++ as first built).** Ticks land at 0, 52, 104, 156, ... ms. After 10 ticks the server is 20 ms behind the ideal schedule, and after one second it has completed only 19 ticks.
- **Timetable (ENet, and the C++ fix).** Ticks are aimed at 50, 100, 150, ... ms. A late tick is followed by an on-time one, so after one second exactly 20 ticks have been completed.

7.2.5 The code behind it

Figure 17 shows the real tick check from the first version of the C++ server and Figure 18 explains it in pseudocode. The line that causes the drift is `last_tick = now;`.

```
// current code (server/main.cpp): the tick loop
auto   now     = Clock::now();
double elapsed = std::chrono::duration<double>(now - last_tick).count();
if (elapsed >= TICK_SEC) {      // 50 ms or more have passed
    last_tick = now;            // restart the timer at "now": the delay is lost
    tick_n++;
    // ... drop silent clients, broadcast snapshots ...
}

// fix (not applied): advance the timer by exactly one tick
last_tick += std::chrono::duration_cast<Clock::duration>(
                 std::chrono::duration<double>(TICK_SEC));
```

Figure 17: The tick check in the C++ server and the one-line change that would remove the drift.

```
STOPWATCH (the C++ server today)     TIMETABLE (Godot timer / the fix)
-------------------------------     ---------------------------------
every loop:                          every loop:
  if now - last_tick >= 50 ms:         if now - last_tick >= 50 ms:
    last_tick = now                      last_tick = last_tick + 50 ms
    do the tick                          do the tick

Stopwatch: noticed at 52 ms -> next tick due at 52 + 50 = 102 ms (drifts).
Timetable: noticed at 52 ms -> next tick due at 50 + 50 = 100 ms (on schedule).
```

Figure 18: Pseudocode for the two ways of timing a tick. Think of a bus that should leave every 50 minutes: the stopwatch driver restarts his watch whenever he notices the bus is due, so any lateness is kept; the timetable driver follows fixed times, so lateness is not kept.

7.2.6 The fix

Two changes were made in `server/main.cpp`. First, the next tick is scheduled at an exact multiple of 50 ms after the previous due time, so lateness does not add up. Second, the wait for packets now ends when the next tick is due (at most 5 ms), so ticks are noticed sooner. If the server ever falls more than a whole tick behind, it resynchronizes instead of firing a burst of ticks. Figure 19 shows the real code, including the resynchronization check.

```
// fixed code (server/main.cpp, excerpts)

// before the loop: the tick schedule
const auto tick_period = std::chrono::duration_cast<Clock::duration>(
                             std::chrono::duration<double>(TICK_SEC));
auto next_tick = Clock::now() + tick_period;

while (true) {
    // sleep until the next tick is due (at most 5 ms) so ticks are not late
    auto until_tick_us = std::chrono::duration_cast<std::chrono::microseconds>(
        next_tick - Clock::now()).count();
    if (until_tick_us < 0)    until_tick_us = 0;
    if (until_tick_us > 5000) until_tick_us = 5000;
    // ... wait for packets for until_tick_us, then read every waiting packet ...

    // 20 Hz tick
    auto now = Clock::now();
    if (now >= next_tick) {
        next_tick += tick_period;
        if (next_tick <= now)            // fell more than a tick behind: resync
            next_tick = now + tick_period;
        // ... tick: drop silent clients, broadcast snapshots ...
    }
}
```

Figure 19: The fixed-timestep tick schedule in the C++ server (excerpts from `server/main.cpp`). The key change is `next_tick += tick_period`: the next tick is due one period after the previous due time, not after the moment the tick was noticed.

Before (Section 7.2.5): stopwatch

```
wait_for_packets(5 ms);
now = Clock::now();
if (now - last_tick >= tick) {
    last_tick = now;
    tick_n++;
    // drop silent clients,
    // broadcast snapshots
}
```

After (Section 7.2.6): timetable

```
wait_until(next_tick, 5 ms);
now = Clock::now();
if (now >= next_tick) {
    next_tick += tick;
    if (next_tick <= now)
     next_tick = now + tick;
    tick_n++;
    // drop silent clients,
    // broadcast snapshots
}
```

removed changed new

Figure 20: The old and the fixed tick loop side by side (simplified from Figures 17 and 19). The red line is the one that causes the drift: it restarts the stopwatch at the moment the tick was noticed. The green lines schedule the next tick from the previous due time, and resynchronize if the server falls a whole tick behind.

7.2.7 Result of the fix

In a 30-second local test the original server ticked at 19.67 Hz and the fixed server at 20.000 Hz. How this changes the cloud results will be reported with the corrected runs.

7.3 Problem 2: snapshots of different size

Both servers send every client a snapshot of all players, 12 bytes per player, plus a header. The C++ header was 16 bytes and the ENet binary header only 8, so each C++ snapshot was 8 bytes larger. The difference is tiny (Table 7) but it is an unnecessary difference between the two backends, so the ENet header was padded to 16 bytes (bytes 8 to 15 are reserved and zero; the first 8 bytes are unchanged). Both transports now send 16 + 12N bytes.

Players N	C++ (bytes)	ENet before (bytes)	ENet now (bytes)
100	1,216	1,208	1,216
200	2,416	2,408	2,416
300	3,616	3,608	3,616

Table 7: Size of one snapshot sent to one client.

7.4 Measurement quirks noticed

- **Slow ENet states.** When ENet was overloaded, a "10 s" ladder state actually lasted about 27 s, because the harness waited for slow client logs. Rates are computed over the real length, so they are correct.
- **The last ladder state.** The last state (300 clients) has no later state to measure its length from, so the nominal 10 s was used while its counters covered about 27 s. This inflated its snapshot rate (up to about 30 per second, which cannot be real at 20 ticks per second).

These are noted and will be handled in the analysis of the corrected runs.

7.5 What changes in the corrected runs

- The C++ server ticks at 20 Hz (fixed-timestep schedule).
- Both servers send 16 + 12N byte snapshots.
- 8 progressive-ladder runs and 8 hold runs per backend will be made again with both patches applied.

Status (22 October 2026): the two fixes are written and tested locally, and a small cloud test is running. The new data, with both patches applied, will be made next (8 progressive-ladder runs and 8 hold runs per backend), and the first-round data in Section 8 will then be replaced.

8 Results

8.1 Test 1, progressive ladder: granular runs (both patches applied)

TODO: interpret the ladder results (latency, packet gaps, client limit, tick rate).

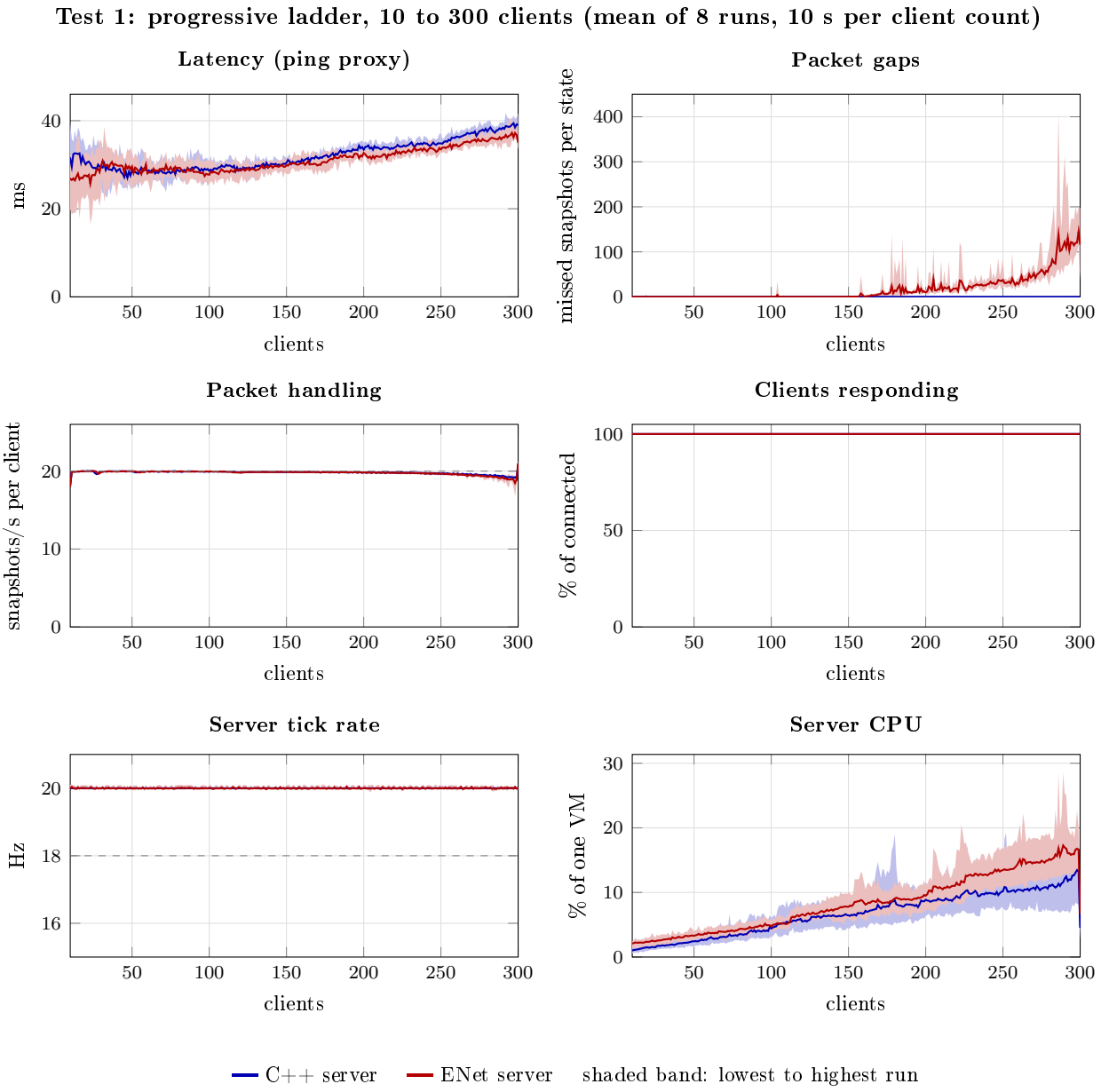


Figure 21: Test 1, progressive ladder (granular batch, both patches applied; server on an e2-standard-2 VM, three n2-standard-4 client VMs, 10 s per client count; the 1-client warm-up state is left out). Each line is the mean of 8 runs and the band spans the lowest to the highest run. Ping is measured by the Godot client (time from its last send until a snapshot arrives), not a true round trip. Packet gaps are skipped snapshot numbers summed over all clients in a state. Dashed lines: 20 snapshots per second and the 18 Hz tick threshold.

Backend	Clients	Resp. (%)	Ping, ms	Snap/s	Gaps	Tick (Hz)	Srv CPU (%)
C++ server	100	100.0	29 (26–31)	19.9 (19.9–20.0)	0 (0–0)	20.00	4.4
C++ server	150	100.0	30 (29–32)	19.9 (19.8–19.9)	0 (0–0)	20.00	6.4
C++ server	200	100.0	33 (32–35)	19.8 (19.7–19.9)	0 (0–0)	20.00	8.9
C++ server	250	100.0	35 (33–37)	19.6 (19.5–19.9)	0 (0–0)	20.00	10.2
C++ server	300	100.0	39 (37–42)	21.0 (20.8–21.4)	0 (0–0)	20.00	4.5
ENet server	100	100.0	28 (25–30)	19.9 (19.9–20.0)	0 (0–0)	20.01	4.7
ENet server	150	100.0	30 (28–31)	19.9 (19.9–19.9)	0 (0–1)	20.01	7.8
ENet server	200	100.0	33 (32–34)	19.8 (19.8–19.9)	10 (7–16)	20.00	9.5
ENet server	250	100.0	33 (33–34)	19.7 (19.5–19.8)	38 (20–85)	20.00	13.5
ENet server	300	100.0	35 (33–37)	21.0 (20.9–21.1)	116 (35–197)	20.00	6.7

Table 8: Ladder results at selected client counts, summed up over 8 runs (10 s per state). Ping, Snap/s and Gaps show the mean and, in brackets, the lowest and highest run. Resp. = clients responding (mean); Ping = the Godot client's ping proxy; Snap/s = snapshots received per client per second; Gaps = skipped snapshot numbers in that state; Tick = server ticks per second; Srv CPU = server CPU.

	C++ server			ENet server		
Run	Passed	Gaps	First fail	Passed	Gaps	First fail
1	292	0	–	291	4,454	–
2	292	0	–	291	3,903	–
3	291	0	–	288	5,402	294
4	291	0	–	291	7,031	–
5	291	0	–	292	4,822	–
6	291	0	–	291	4,413	–
7	292	0	–	291	5,980	–
8	291	5	–	292	4,852	–

Table 9: Per run, over the whole ladder (292 states): states that met the health limits, snapshot gaps summed over all states, and the first client count (from 10 upward) at which a state failed (– = none).

8.2 [SUPERSEDED] Test 2, hold at 300 clients: ramp-up and hold

SUPERSEDED. First-round data, kept for reference. Replaced by the granular runs in Section 8.1; the hold tests will be redone with the corrected servers.

Figure 22 is the hold test at 300 clients, the heaviest target, summed up over 8 runs. The grey left part is the ramp-up, when clients are being added (plus a short wait); time 0 is the moment the hold starts and no more clients are added. What matters is whether the lines recover after the ramp-up. ENet starts to fall behind during the ramp-up and does not recover: during the hold on average about 75% of its clients respond (61 to 88% across runs), it delivers about 16 snapshots per client per second, and its ping is around 1.4 s.

8.3 [SUPERSEDED] Test 2, hold at every target

SUPERSEDED. First-round data, kept for reference. Replaced by the granular runs in Section 8.1; the hold tests will be redone with the corrected servers.

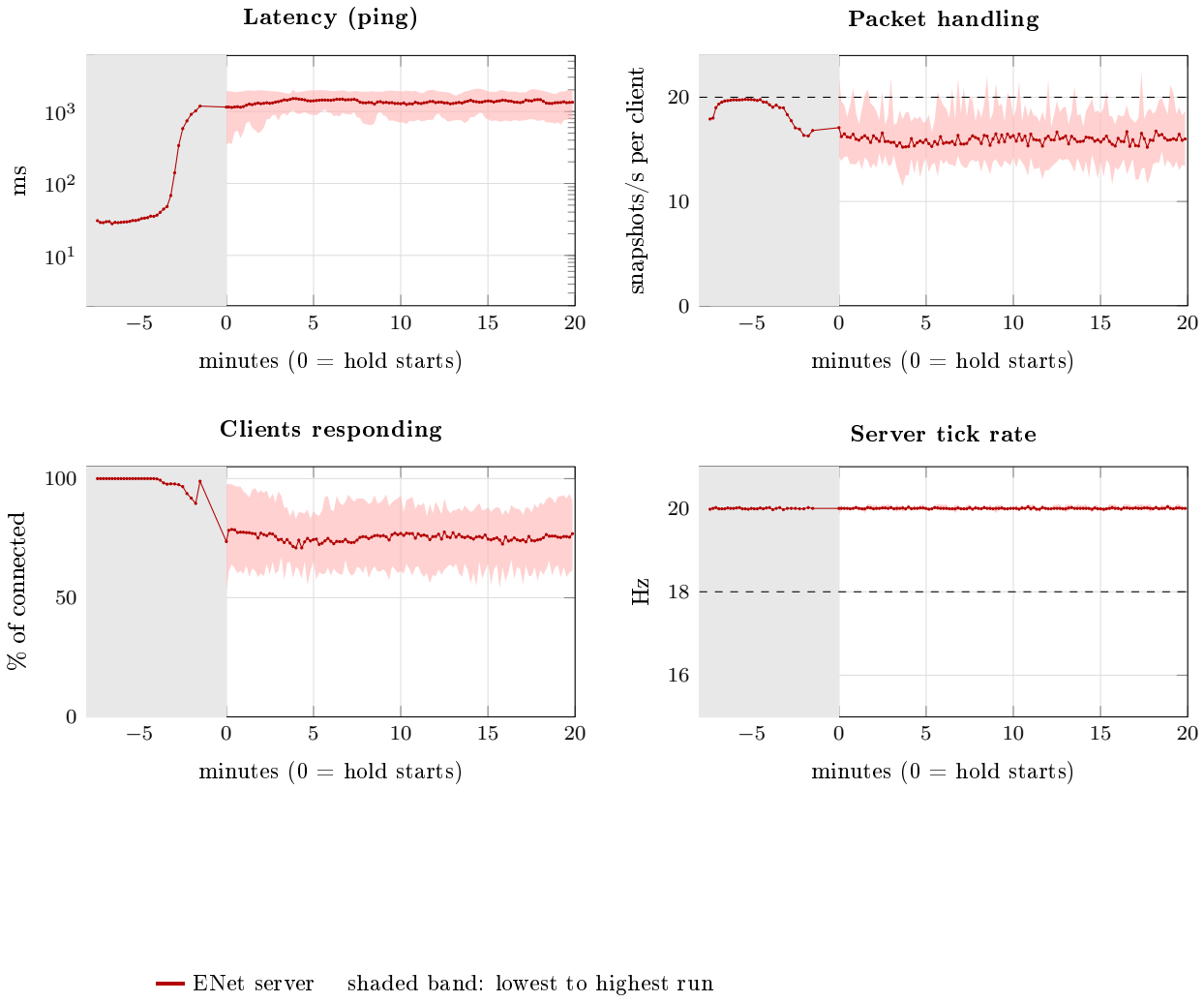


Figure 22: **[Superseded]** Test 2, hold at 300 clients, one point per 10 s. Each line is the mean of 8 runs and the band spans the lowest to the highest run. Grey: clients being added (ramp-up and a short wait before the hold). White: the 20-minute hold. Dashed lines: 20 snapshots per second and the 18 Hz threshold.

Figure 23 repeats the hold test for all four targets, again with the ramp-up on the left, and Table 10 sums up each target. ENet held 180 clients in all eight runs, but only one of eight runs at 260 clients and one of eight at 270, and none at 300.



Figure 23: **[Superseded]** Test 2, share of clients responding during the hold at every target (mean of 8 runs). Grey: ramp-up. The panels show Godot ENet at 260, 270 and 300 clients; the line is the mean of 8 runs and the red band spans the lowest to the highest run. ENet at 180 clients also stays at 100% in all 8 runs and is not shown.

Backend	Clients	Resp. (%)	Snap/s	Ping, ms	Tick (Hz)	Held
Godot ENet	180	100.0 (100.0–100.0)	20.0	36 (35–36)	20.0	8 of 8
Godot ENet	260	93.2 (87.8–100.0)	19.3	701 (106–1267)	20.0	1 of 8
Godot ENet	270	86.9 (76.7–100.0)	18.5	927 (113–1612)	20.0	1 of 8
Godot ENet	300	75.2 (61.1–88.4)	15.9	1356 (924–1908)	20.0	0 of 8

Table 10: **[Superseded]** Hold results at each target, summed up over 8 runs. Each run is first averaged over its 20-minute hold; the table gives the mean of those averages with the lowest and highest run in brackets. Resp. = clients responding, Snap/s = snapshots received per client per second, Tick = server ticks per second. A run "held" if its hold averages met the health limits (at least 99% responding, at least 18 snapshots per second, ping at most 150 ms, tick at least 18 Hz).

8.4 [SUPERSEDED] Example data

SUPERSEDED. First-round data, kept for reference. Replaced by the granular runs in Section 8.1; the hold tests will be redone with the corrected servers.

The tables below are real rows from our runs, to show what the data looks like.

Progressive ladder: 8 runs summed up. Each row sums up the 8 complete ladder runs at one client count: the mean, with the lowest and highest run in brackets. Resp. = clients responding (mean), Snap/s = snapshots received per client per second, Tick = server ticks per second, Passing = runs that met the health limits at that count.

Backend	Clients	Resp. (%)	Ping, ms	Snap/s	Tick (Hz)	Passing
Godot ENet	100	100.0	29 (28–31)	19.9 (19.8–19.9)	20.0	8 of 8
Godot ENet	150	100.0	33 (31–34)	19.6 (19.3–19.9)	20.0	8 of 8
Godot ENet	200	100.0	38 (34–42)	18.0 (16.5–20.2)	20.0	3 of 8
Godot ENet	250	100.0	478 (46–942)	14.4 (12.8–17.6)	20.0	0 of 8
Godot ENet	300	99.3	252 (50–1474)	31.2 (28.2–35.8)	20.0	3 of 8

Table 11: **[Superseded]** Example ladder rows, summed up over 8 runs (16–17 October, 10 s per state). Ping and Snap/s show the mean and, in brackets, the lowest and highest run.

One thing to note. The ENet server's tick rate stays near 20 Hz even when its clients are failing, so for ENet the breakdown shows up as lost clients, low snapshot rate and high ping rather than a tick-rate drop. TODO: confirm why. The ENet server counts ticks from a Godot timer that fires every 50 ms without drift, so its tick counter keeps pace with the clock whether or not the work finished in time.

8.5 How the runs were summed up

Both tests combine 8 complete runs per backend. For the ladder, each of the 292 client counts is paired with the same count in the other runs. For the holds, each 10 s state is paired by its position in the ramp-up or hold. The mean, lowest and highest value of each measure are reported. A ladder run passes at a client count if that count met the health limits; a hold run held if its hold averages met them. Hold runs 3 and 4 were missing some ENet captures because those captures failed, so they were repeated in a separate make-up batch and used in their place (ENet 270 and 300 clients for run 3, ENet 300 clients for run 4).

9 Discussion

TODO.

10 Conclusion

TODO.

References

[1] L. Aukstakalnis. *Server-Client Networking Implementation Using the Godot Game Engine*. 2024.

[2] F. T. Al-Dhief et al. Performance comparison between TCP and UDP protocols in different simulation scenarios. *International Journal of Engineering & Technology*, 7(4.36):172–176, 2018.

[3] C. Jones. Go UDP programming: a beginner-friendly guide to building fast, real-time apps. DEV Community, 2025.

[4] E. Retunsky. Benchmarking low-level I/O: C, C++, Rust, Golang, Java, Python. Star Gazers, 2023.

[5] Godot Foundation. Exporting for dedicated servers. *Godot Engine Documentation*, 2026.

[6] Godot Foundation. UDPServer. *Godot Engine Documentation*, 2026.

[7] Unity Technologies. Ticks and update rates. *Netcode for GameObjects 2.5 manual*. `https://docs.unity3d.com/Packages/com.unity.netcode.gameobjects@2.5/manual/learn/ticks-and-update-rates.html`

[8] Valve Developer Community. Source Multiplayer Networking. `https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking`